

Node.js Interview Preparation (4+ Years)

Mock Interview Questions with Sample Answers

Q: What is the difference between `process.nextTick()` and `setImmediate()`?

A: `process.nextTick()` executes after the current operation, before I/O events. `setImmediate()` runs on the next event loop cycle. Use `nextTick` for priority tasks, `setImmediate` for deferring execution.

Q: How do you handle errors in asynchronous code?

A: Use callbacks with error-first pattern, promises with `.catch()`, or `async/await` with `try/catch`.

Example:

```
try { await fn(); } catch(err) { console.error(err); }
```

Q: How would you structure a Node.js project?

A: Separate routes, controllers, services, models, and middleware. Use a layered architecture to keep code maintainable and testable.

Q: How do you implement caching in Node.js?

A: Use in-memory cache like `node-cache` or external like Redis. Cache data to reduce DB calls.

Q: How do you handle authentication in Node.js?

A: Use JWT tokens. Store in headers/cookies. Validate with middleware.

System Design Challenges

Q: Design a URL Shortener Service

A: - Endpoints: `/shorten`, `/:code`

- Store mappings in DB
- Generate shortcode using hash
- Handle duplicates and expiration

Q: Design a Real-Time Chat App

A: - Use WebSocket or Socket.io

- Store messages in MongoDB

- Redis for pub/sub
- Authenticate socket connection

Q: Build a Rate Limiting Middleware

A: - Track IP request count

- Store in memory or Redis
- Reset counts periodically
- Return 429 when limit exceeded

Q: Design an API Gateway in Node.js

A: - Use Express

- Route to internal services
- Add auth, logging, throttling in middleware

Coding Tasks for Node.js

Q: Custom Logging Middleware

```
A: function logger(req, res, next) {  
  console.log(`${req.method} ${req.url}`);  
  next();  
}
```

Q: Promise Wrapper for fs.readFile

```
A: const fs = require('fs').promises;  
  
async function readFileData(filePath) {  
  try {  
    return await fs.readFile(filePath, 'utf8');  
  } catch (e) {  
    throw e;  
  }  
}
```

Q: Throttle Requests (per user/IP)

A: const limits = {};

function limiter(req, res, next) {

const ip = req.ip;

const now = Date.now();

if (!limits[ip]) limits[ip] = [];

limits[ip] = limits[ip].filter(t => now - t < 60000);

if (limits[ip].length >= 10) return res.status(429).send('Too Many Requests');

limits[ip].push(now);

next();

}

Q: Stream Large File to Client

A: app.get('/download', (req, res) => {

const stream = fs.createReadStream('bigfile.zip');

stream.pipe(res);

});